

GenAI Report

Sondre Losvar, Roberto Neglia

May 2025

Contents

1	Introduction	2
2	Pipeline	3
2.1	LLM	4
2.1.1	Training data	4
2.1.2	Train process	5
2.1.3	Running the LLM	6
2.2	Image Generator	7
2.2.1	Training data	7
2.2.2	Train process	9
2.2.3	Running the Image Generator	11
2.3	Putting everything together	11
3	Setup	11
4	Examples	13
5	Conclusions and Future Work	15
A	Appendix	19

1 Introduction

In this paper we are going to present and discuss our project in INF-3993, Generative artificial intelligence. The goal of this project is to utilize some of the topics that have been covered in this course.

When deciding what we were going to make, we took great inspiration from a video from Yannic Kilcher's youtube channel [1].

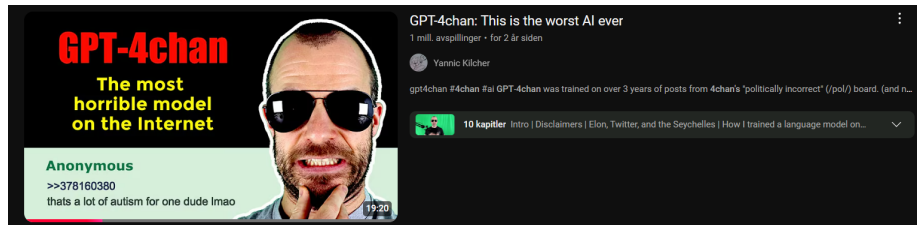


Figure 1: Yannic Kilcher's video of training a LLM on 4chan posts

In this video Yannic makes a fine-tuned model using posts from /pol/ (Politically Incorrect) of the 4chan web forum. 4chan is an imageboard website where users can post images and comments on various topics, and /pol/ is one of the boards on 4chan that focuses on political discussions. He used this model to make numerous posts on /pol/ and observed how the community reacted. One of the major indicators of this model being a bot is that it cannot post or interpret images.



Figure 2: Pepe the frog

Our idea was then to try to improve this model by making an image generator that can generate a Pepe the frog image to accompany any post. See Figure 2 for an example of a Pepe image. Pepe the frog is a popular meme on 4chan and

is often used to express various emotions and sentiments, making it a suitable choice for our project.

During the years, Yannic’s model has been taken down from the HuggingFace repository because of the controversial nature of the content it was trained on and that it could produce. Therefore, we also have to replicate his model by using the same dataset he used. The dataset and model he made was way too computationally demanding than what we can perform, so we planned to use a base model with a Low Rank Adaptation (LoRA) or a few-shot learning approach to try to replicate his model.

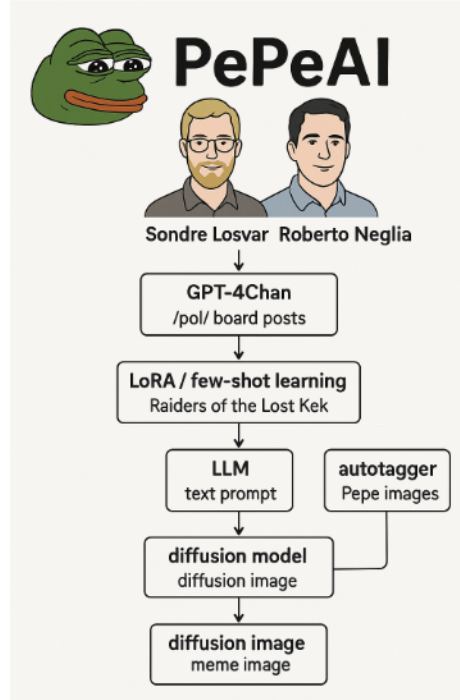


Figure 3: Pipeline of the pitched idea

We decided to further scale down the model by only teaching it to generate opening posts (OPs) with an associated Pepe image. We planned to train the image generator by using Pepe images and a LoRA on a diffusion model that does not know what Pepe is. Then reduce the LLM’s output to something we can use to prompt the image generator. The pipeline of this idea can be seen in Figure 3.

2 Pipeline

In this section we will describe the pipeline of our project, which consists of three main components: a Large Language Model (LLM), an image generator,

and a Graphical User Interface (GUI) that ties everything together. The LLM is responsible for generating opening posts (OPs) based on user input, while the image generator creates Pepe images to accompany the generated OPs. The GUI allows users to interact with the system and view the generated content.

2.1 LLM

The Large Language Model (LLM) is a crucial component of our project, as it is responsible for generating opening posts (OPs) based on user input.

2.1.1 Training data

The training data we are going to use to train the LLM is called “Raiders of the Lost KEK” and it contains 4chan /pol/ posts that spans over a period of almost 3.5 years, which leads to about 135×10^6 posts over 3.4×10^6 threads[2]. This file is downloaded as a .tar.zst file and is 24 GB, when it is unpacked into a .ndjson file it is about 106 GB. This is more than we are able to utilize, so we are just using a subset of this data. This file contains a lot of different variables for each /pol/ thread, which can be viewed in the data’s documentation[2]. We are trying to learn to make opening threads, so the variables we are interested in are just the subjects and starting comments on each thread. The opening threads are in an HTML format and contain URLs and needs to be cleaned up, while the subject is just a simple sting. After cleaning we can construct the training prompts for the LLM training and end up with the format:

```
Generate a /pol/ style opening post about: {subject}
{opening post}
```

For an LLM to use this for training we have to apply a tokenizer, converting this string into a set of number representation. Then we can apply an attention masking to the tokenized representation and end up with the format:

```
input ids: [32, 4983, 2, ..., 0, 0]
attention mask: [1, 1, 1, ..., 0, 0]
labels: [32, 4983, 2, ..., 0, 0]
```

Our training data has several text formats, we have common text (e.g. hi Anon. i think the election ...), image or video centered text (e.g. Look at this: {media} I cant believe ...), and we have greentext which is a story teller mode in the format that is used on 4chan, which is often used to tell a story in a humorous way. The greentext format is characterized by the use of the >-character at the beginning of each line, which is used to indicate a new line in the story. An example of greentext format, which is a common way to tell stories on 4chan, can be seen in Table 1.

We are not sure how the LLM will react to these different types, so we feed it all of it. We expect that this might introduce some noise into the training. We have one step to mitigate this kind of noise, which is to require a minimum number of characters after cleaning. This aims to remove the image or video centered text with no content that direly refers to the image or video.

```

> be me. ML student
> run LLM_training.py
> watch it load, crack open countless beers
> ...
> training done. 36 hours later
> hard days work
> feels good man

```

Table 1: Example of greentext format.

2.1.2 Train process

We first need to select a base model that we are going to apply the LoRA to, we have quite some freedom here. We want a large and good, public model while being limited to common hardware. We opted for the Microsoft Phi-2 model[3], since it is well known, readily available and most importantly runs on our local GPU (an NVIDIA RTX 3070).

The next step is then to select how large or deep the LoRA is going to be, which depends on the complexity of our data and the data discrepancy between the base model’s training data and our data. We expect the model will relatively easily be able to adapt to the general language usage. The hard thing to learn is all the general vocabulary of 4chan, like topic, phrases, movements, ideologies (e.g. QAnon, glowie, ZOG, blue-red-blackpilling). We don’t expect to learn all to the topic, just mimic some of their usage. We reckon therefore that we have to scale the LoRA’s rank with the size of the data subsample. We ended up with setting the LoRA rank to 12 and we can therefore compute the amount of trainable parameters as in [4]:

$$|\Theta| = 2 \times \hat{L}_{LoRA} \times d_{model} \times r = 7864320 \approx 7.9 \times 10^6 \quad (1)$$

where $L = 32$ is the number of layers in each module of the LLM, $M = 4$ is the number of modules, $\hat{L}_{LoRA} = M \times L = 128$ is the number of layers in the LoRA, $d_{model} = 2560$ is the hidden size of the transformer model within the LLM, and $r = 12$ is the rank of the LoRA.

The base model has about 2.7×10^9 parameters, which means we train about 0.29% of the parameters of the hole model. The LoRA has a scaling variable (LoRA_alpha) for the training parameters which determines how much the LoRA is unitized in making predictions. This is scaled with the rank of the LoRA and is set to $r \times 3.5 = 42$.

To train this model effectively we have to use a quantizer like Bits And Bytes. This functions to reduce the memory usage of the model and make the LLM run over $\times 10$ faster on a RTX 3070 with 8 GB VRAM. The default model loads the weights in 32-bit precision, with this quantizer we load the weights with 4-bit precision and computes in 16-bit precision, which is an 8 fold reduction of memory usage. In addition to this it uses a 4-bit data type called NormalFloat4 (nf4), which is supposed to work well with LLMs. The trade off with this is a lower precision, which might degrade the max potential of the training.

We also have to set up training arguments for the model. We load the data in batches and assign them to each device(GPU). This is limited by the GPU memory, we trained with a 16 GB VRAM GPU through Sigma2 and are then able to have a batch size of 25. We could also use gradient accumulation, which only updates the weights every n passes. This makes the training slightly faster and smoother. This would still require just as much VRAM s.t. batch size $\times n = 25$. We have to select an optimizer, we use a slightly modified Adam optimizer. We use 8-bit AdamW with paging, 8-bit precision to conserve memory usage, paging is a way to include CPUs if the GPU runs out of memory, The W stands for weight decay and makes the loss surface smoother and gives a smoother convergence. We also need a way to see the progress of the training to ensure that the training does not die. We then set some data as test data and use the built-in logging function and give training feedback every 50 batches. We then set the training epochs to 3, passing the data 3 times though the training. 3 epochs might be a bit low, but since we have so much data that we want to use, it seems justified.

At this point we are approaching a bottleneck with how we load and clean the data. This can be fixed by dividing the data into chunks and divide it across multiple CPUs. This has been implemented and scrapped and is now just partially implemented with having 4 CPU workers set in the training arguments. This has been scrapped. since it is unnecessary, it is not a significant training speed improvement and would complicate the code significantly. If we were to scale up the training with more data and more or bigger GPUs this might be necessary.

With all of this in the box we are ready to start training. Our final model was trained for about 36 hours. During training we see that the cross entropy loss measurement goes from about 3.5 from the first log to about 2.6 after just a few logs and is mostly flat across the training and creeps towards 2.3 in the end of the training and would be a very flat and boring graph if we had it.

2.1.3 Running the LLM

We have trained several smaller models and after training on 1000 posts it seems to perform fairly well and are able to copy the tone of the 4chan posts with a very casual tone with political, vulgar or racial comment. This seem to be just a superficial facade since often it made very generic "LLM speak" posts and if it was prompted with a subject the base model would not have prior knowledge about it would fail miserably(e.g. Qanon, Glowies, pizzagate). The final model was trained on 200 000 posts for 36 hours and is noticeably better, it still has apparent flaws, but given a few tries it is able to make a post that might be thought to be a real /pol/ post for even theses harder subjects.

One of the most apparent mistakes or oversights is the misplacement of the colon when prompting. Currently the model is trained and prompted with:

```
Generate a /pol/ style opening post about: {subject}
{opening post}
```

The intended prompt should be:

```
Generate a /pol/ style opening post about {subject}:  
{opening post}
```

This minor oversight results in the model not separating and discriminating the prompted subject from the post text. The model is now able partially prompt itself by writing out the subject (e.g. cats → cats VS {ethic group}, politics → politics and culture). This also results in the model very often fusing the subject and post, and some times it has this disconnected subject from the post. We are not sure if this is a bug or a feature. The upside with this is that it turns a subject that you would not find on /pol/ to something you might find there and steers the LLM towards its intended usage. The downside with this is less control over the LLM and a less predictable output format.

Because of this misplaced colon we just remove the starting part of the prompt after generating (“Generate a /pol/ style opening post about:”). Most of the time this works fine and it generates a post by starting the sentence with the subject, and sometimes it starts with the subject which is disconnected from the post.

The final LLM generator has a max token length of 150, giving a moderate length /pol/ post. The generator has the top k variable set to 20 and temperature set to 1. These variables affect how it selects from the next token prediction, top k means it only has the 20 most likely tokens in its draw pool, and temperature is a scaling of the draw probabilities. Where a temperature of 0 means that it will draw the most likely token 100% of the time, while a very high temperature makes the top 20 token close to equiprobable to draw. The generator has also been set to penalize repetitions. Without this had a decent chance to just loop the same text, often like racial chants. The downside with this penalization is that it seems to kill the generator’s attempts to make greentext.

The greentext we observed before this penalization seems to have some potential, but was not able to use the format in a consistent manor, going back and forth from normal text and greentext. The image and video centered post seems to have made an impact on the generator and it might add “pic related” or “pic unrelated”.

2.2 Image Generator

The image generator is another key component of our project, as it is responsible for creating Pepe images to accompany the generated opening posts (OPs).

2.2.1 Training data

To adapt the image generator to our task (i.e., generating Pepe images), we first had to find a suitable dataset of Pepe images. This dataset needed to be diverse enough to capture the various expressions and contexts in which Pepe the frog is used.

After some research, we identified several sources of Pepe images, including fan art, memes, and other online content, in this Kaggle dataset ¹, containing over 2,300 Pepe images.

Each image in the dataset is not associated with a specific text prompt, but the image file is named in a way that reflects the content of the image. For example, an image file named

`000007_pepe_crying_closed_eyes_wearing_beats.jpg`

would be a crying Pepe with closed eyes and wearing Beats headphones (Figure 4).



Figure 4: `000007_pepe_crying_closed_eyes_wearing_beats.jpg`

At first, we thought this naming convention provided a useful way to generate text prompts for the image generator, extracting the relevant information from the file names to create text prompts that describe the content of the images. However, as we will also discuss in Section 5, we found that this approach did not work as well as we had hoped.

Hence, we decided to select a small subset of the dataset that we deemed suitable for training the image generator, ensuring that the images were of high quality and diverse enough to capture the various expressions and contexts in which Pepe the frog is used, and captioned the images in a way that reflects the content of the image at a higher level, by expressing the main emotion expressed by the image. This improved a lot the quality of the generated images based on the prompted topic that we used to generate the OPs.

In the final version of the dataset that we used for training the image generator, we manually selected 43 Pepe images, which we found to be sufficient for letting the image generator capture the essence of Pepe the frog and generating high-quality images that align with the desired prompted topics.

¹<https://www.kaggle.com/datasets/tornikeonoprishvili/pepe-memes-dataset>

The dataset is publicly available on HuggingFace ².

2.2.2 Train process

We decided to apply LoRA to the Stable Diffusion 2 base model, which can be publicly accessed on HuggingFace³. This model is a latent diffusion model [5] trained from scratch for 550k steps at resolution 256×256 on a subset of the LAION-5B dataset [6], filtered to remove explicit content, and then further trained for other 850k steps on the same dataset at resolution 512×512.

We used the `diffusers` library [7] to apply LoRA to the Stable Diffusion 2 base model. The `diffusers` library provides a simple and efficient way to apply LoRA to diffusion models, allowing us to adapt the model to our specific task of generating Pepe images.

To do that, we used the `train_text_to_image_lora.py` script available on the `diffusers` GitHub repository under the `examples/text_to_image` directory⁴, which allows us to train a LoRA model on a text-to-image task by simply running the script with the appropriate arguments, such as the name of the base model to be adapted with LoRA, the name of the dataset to be used for training, and the number of training steps to be performed. For all the details on the parameters that we used to train the LoRA model, please refer to table 2.

We decided to use a rank of 16 for the LoRA model, in order to have a good balance between model performance and computational efficiency (the higher the rank, the higher number of trainable weights in the LoRA model). With these parameters, the number of training weights in the LoRA model is 3,319,808, which is approximately 0.38% of the total number of weights in the Stable Diffusion 2 base model, which is 865,910,724.

In this case, we didn't tweak the `LoRA_alpha` parameter, but we left it to be the same as the rank of the LoRA model, which is 16 (i.e., it scales as the rank of the LoRA model).

The number of training steps was set to 4300, which is set in order to let the model train on each image in the dataset 100 times. The training was performed on a single NVIDIA RTX 6000 Ada Generation GPU with 48 GB of VRAM, which allowed us to train the LoRA model with a batch size of 24 images and a gradient accumulation of 4 steps (i.e., "virtually" 96 images per training step) in about 30 minutes, without having to use mixed precision training or quantization, which would otherwise be necessary to fit the model in the GPU memory, with a small performance loss.

The training process was monitored using the `wandb` library [8], which allowed us to visualize the training progress and the validation results in real-time. The validation was performed every 50 epochs, using the prompt

`pepe the frog, happiness`

²https://huggingface.co/datasets/RobertoNeglia/pepe_dataset_sentiment

³<https://huggingface.co/stabilityai/stable-diffusion-2-base>

⁴https://github.com/huggingface/diffusers/blob/main/examples/text_to_image/train_text_to_image_lora.py

Parameter	Value
pretrained_model_name_or_path	stabilityai/stable-diffusion-2-base
dataset_name	RobertoNeglia/pepe_dataset_sentiment
dataloader_num_workers	8
resolution	512
center_crop	-
random_flip	-
caption_column	‘‘features’’
rank	16
train_batch_size	24
gradient_accumulation_steps	4
max_train_steps	4300
learning_rate	1e-4
max_grad_norm	1.0
lr_scheduler	‘‘cosine’’
lr_warmup_steps	0
checkpointing_steps	500
validation_prompt	‘‘pepe the frog, happiness’’
validation_epochs	50
seed	42

Table 2: Parameters used to train the LoRA model.

to see how well the model was able to generate images of Pepe throughout the training epochs.

While the training loss doesn’t show a clear decreasing trend, the validation results showed a gradual improvement in the quality of the generated images, indicating that the model was learning to generate more accurate representations of the target concept. See Figure 5 for a visualization of the generated images at different training phases.

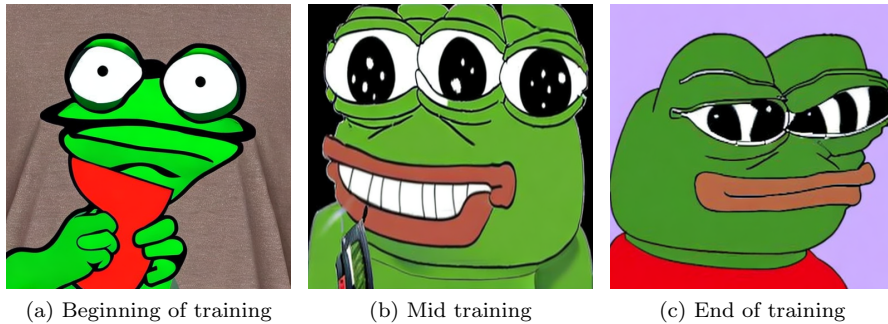


Figure 5: Generated images at different training phases of the LoRA model.

From the generated images, we can see how the base model does not know

what Pepe is, while as the training progresses, the model learns to generate images that resemble Pepe the frog, capturing its essence and expressions.

The final LoRA model was saved and uploaded to HuggingFace ⁵.

2.2.3 Running the Image Generator

The trained LoRA model is used to generate images of Pepe the Frog based on the user’s input topic, also used to generate the opening post (OP). To generate an image, we set the `num_inference_steps` parameter to 100, which means that the diffusion process will perform 100 denoising steps to generate the final image. The `guidance_scale` parameter is set to 10, to give more guidance to the model in generating the image with respect to the text prompt.

Moreover, a negative prompt is used to avoid generating images that are not relevant to the topic; in our case we used “bad quality, low resolution, blurry, out of focus, watermark” as negative prompts.

No other parameters were changed from the default values of the Stable Diffusion 2 pipeline, as the results were satisfactory for the scope of our project.

2.3 Putting everything together

The finished model is run through a Graphical User Interface (GUI). The GUI itself is made using the default python library “tkinter” and is a very basic setup with a scrollable text box, image box, text input box and a generate button.

The user can input a topic in the text input box, and when the generate button is pressed, the LLM is run with the input topic to generate an opening post (OP). The generated OP is then displayed in the scrollable text box. The image generator is then run with again the input topic to generate a Pepe image based on the same topic of the OP. The generated image is then displayed in the image box. The GUI is designed to be simple and user-friendly, allowing users to easily generate opening posts and images based on their input topics.

3 Setup

To run or train this model we need a GPU that is compatible with NVIDIA’s CUDA. To run the model we need to set up a new environment and use the `environment.yml` file in the repository to install all packages [9] required. After setting up the environment and the files from the github repository we can run the `GUI.py` file in the `script` folder, which launches the GUI with a text box we can interact with.

To train the LLM model one can use the same `environment.yml` file used to run it for inference, and download the training data for the LLM. The LLM training might run into compatibility issue between the Bits And Bytes quantizer and the Windows operating system, which can be fixed by using an

⁵https://huggingface.co/RobertoNeglia/pepe_generator_sd2_sentiment

unofficial patch for windows. The training can then be done by running the `LLM_training.py` file and tested at inference time using the `LLM_test.py` file.

Similarly, the image generator training can be run by using the same environment, installing the `diffusers` package from source, export the necessary environment variables, such as the name of the base model to which the LoRA model is applied, the name of the dataset and the output folder for checkpoints. One important thing to note in this case is to login to the Hugging Face hub using the

```
huggingface-cli login
```

command, as the base model (like Stable Diffusion 2 base) may be publicly available but requires authentication to download, and also for uploading the LoRA model to the Hugging Face repository if desired. Moreover, the training can be monitored using the WandB tool, which also requires logging in to the WandB service using the

```
wandb login
```

command.

The training can then be run by executing the `train_text_to_image_lora.py` file with the necessary parameters, as described in Section 2.2.2. To accelerate the training process, we used the `accelerate` package [10], which allows us to run the training on multiple GPUs or even on a single GPU with mixed precision.

More precise setup instructions and necessary files can also be found on the project’s GitHub repository [9]⁶.

⁶<https://github.com/RobertoNeglia/PePeAI>

4 Examples

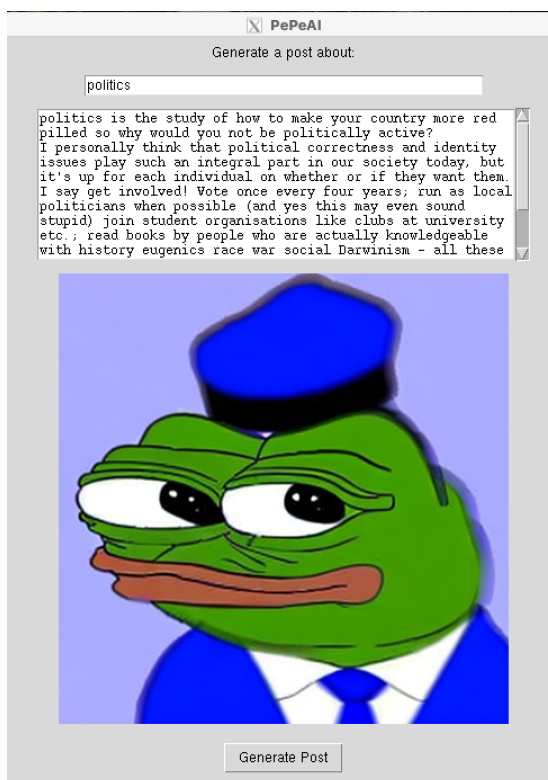


Figure 6: Politician Pepe

In Fig. 6 we can see a good example of a not too edgy post. The text mimics what we might expect from a 4chan post and uses the term “red-pilld” correctly (synonymous with enlightened in this case). The image seems to reflect the text with an image of a levelheaded, intellectual, and steadfast looking Pepe. The text has some minor issues like saying that political correctness is impotent, on a forum called politically incorrect (/pol/), and follow this up with saying that it is impotent to know about race wars, which might be seen as contradictory. Mentioning the importance of political correctness might also be something it has learned and is intended to provoke a reaction from the /pol/ community. The image is a bit blurry, but clearly resembles a politician Pepe dressed in a nice suit and tie.



Figure 7: QAnon Pepe

We see in Fig. 7 an example of a post generated with a more niche and edgier subject. This is also reflected in the image generator by the poorly generated generic Pepe with multicolored eyes. The text is incoherent but is on the right topic with references to pizza (pizzagate), Wikileaks, justice (persecution), Trump and Hillary. The post praises the downfall of Wikileaks, which is in contradiction with what any QAnon poster would do. The LLM seems to attempt to start to generate greentext with “>” and might be a big factor in why this text is so poor. QAnon posts are also known to use a lot of niche or cryptic language, this might also be a big factor for why the text is so incoherent and of poor quality.

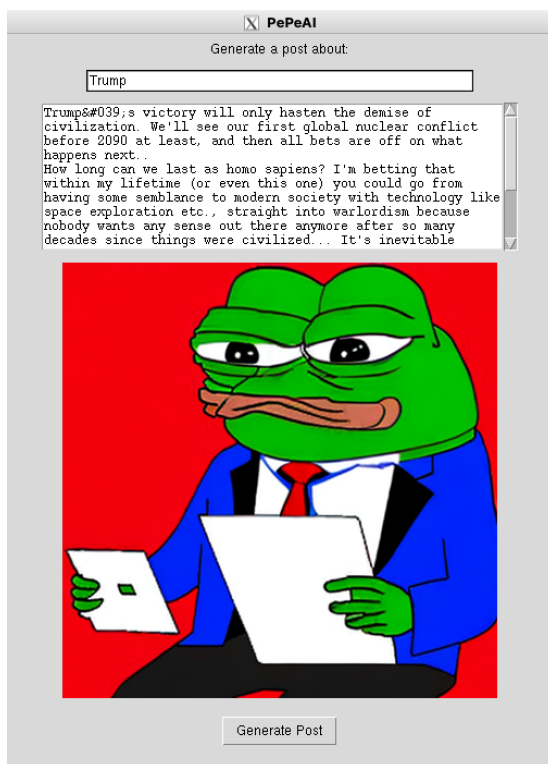


Figure 8: Trump Pepe

Lastly, we see in Fig. 8 another good example of a generated post and Pepe. The text is coherent with the prompted topic, looks like a post that could be made on /pol/ and the image is a nice generated Pepe that looks like a politician.

We also include other examples of generated posts and images in Appendix A.

5 Conclusions and Future Work

The LLM has some glaring issues. The aforementioned misplacement of the prompted colon in Section 2.1.3 is one of the simplest issues to fix. This can be fixed by moving the colon to:

Generate a /pol/ style opening post about \{subject\} :
\{opening post\}

Or, if we wanted to separate it further, we could insert some spacing text like:

Generate a /pol/ style opening post about \{subject\} in 3. 2. 1.:

\{opening post\}

This makes it so that the generator can't predict and continue writing on the inserted subject. This also ensures a consistent output format.

However, a drawback of this is that, if we were to prompt it with a subject that is not /pol/ related, the model might not be able to generate posts on arbitrary topics. In this case, if we were to insert a non-/pol/ subject as an input, the LLM might either generate a post that is not /pol/-related, or generate a post that is /pol/-related but not coherent with the subject, deviating from the input of the user and arriving by a roundabout way to the subject.

The LLM also seems to have learned a bit of noise from poorly cleaning the training data. We know that there are residuals from URLs and maybe some HTML code left in the data, and the model might generate some residual URLs or HTML start characters, as seen in Fig. 8 with the residual `'` for the apostrophe in "Trump's".

The training data also has a spam measurement [2] for each thread that we could utilize to further clean the text. This would also hopefully fix the issue of the LLM repeating itself. Then we could remove the penalization for repetition, which would probably improve the generation and bring back it's attempts to generate greentext, preventing the issues seen in Fig. 7 and also generating more valid greentext posts.

Another issue which is harder to address are these image and video centered posts. This makes the model refer to content that is not there and makes vague text with low content. We are not sure how to filter out or identify these kinds of posts by conventional means.

Along with the aforementioned fixes, the model might be significantly improved by adjusting the generation parameters like temperature, top-k and top-p.

The LLM model can be expanded on in several ways. These threads or opening posts we have been training on also has comments which can be used to make the model into a proper chat bot. The training and setup of this chat bot might need to be structured in a clever way, since on a thread we have comments to the thread itself and direct replies and branches of replies to replies. This makes a sort of nested subsections of conversations that has to be sorted out, and is slightly more complicated than training a common chat bot.

The training data also has a toxicity score and server toxicity score, which would be interesting to use [2]. The measurement itself comes from a machine learning tool from Google's Perspective API. Replicating this tool might be complicated, but we could use the provided scores to either clean the data or train the model to be more or less toxic. The simplest way to implement this would be to insert this score in the training prompt:

Generate a /pol/ style opening post about {subject} with a toxicity of {toxicity score} :
{opening post}.

A final consideration with respect to the LLM is the generation of hateful content. Since we haven't removed any kind of potentially controversial content

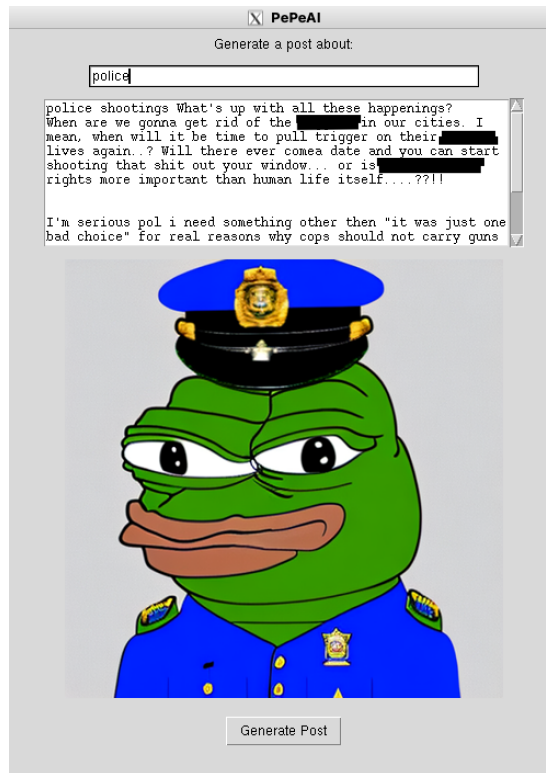


Figure 9: Hate speech in generated post

from the dataset, the model generates posts that often contain hate speech and other offensive content, as can be seen in Fig. 9.

This is also a result that Yannic’s model achieved, and it’s also probably why his model is not publicly available anymore. We have not implemented any kind of hate speech detection or filtering in our model, but it is something that could be done in the future. This could be done by using a hate speech detection model, to filter out posts that contain hate speech, or, as already mentioned, using the toxicity score from the training data to filter out even more posts that are too toxic.

This is also why, after the project will be finished and evaluated, we will not make the weights of the model publicly available, as we do not want to contribute to the spread of hate speech and other offensive content.

Regarding the image generator, one big issue that we faced was the lack of real connection between the generated text and the generated image. We first tried by using the whole dataset available of 2300 images of Pepe, with the name of the image as a caption. This resulted in the generator not being able to generate images that were coherent with the /pol/ posts, as the generator was not able to learn the connection between the generated text (or the input topic)

and the generated image. This can be fixed by cleaning the training data further, captioning the images with some relevant generated text, or using a different training method such as the one DreamBooth uses [11], which allows us to train the model on a specific subject and generate images that are coherent with the subject in a specific style or context. We tried to explore this by using a subset of the dataset with more relevant captions and fine-tuning the model on this subset, but results were not satisfactory because of the lack of real connection between the training images and the generated post.

We also tried to use a sentiment analysis model to infer the sentiment of the generated post and use this sentiment to generate an image that reflects the sentiment of the post. This was not successful, as at that point the image generator was just generating images that were coherent with the sentiment of the post, but not with the topic of the post.

At the end, we decided to just feed the LoRA adapted Stable Diffusion model with the user input topic, which resulted in images that were coherent with the topic of the post most of the time.

The image generator can be improved by using a larger dataset of Pepe images with better captions, and let the model train for longer, and using a more powerful base model as a starting point.

References

- [1] Yannic Kilcher. GPT-4chan: This is the worst AI ever. <https://www.youtube.com/watch?v=efPrtcLdcM&t=10s>, Release: 03.06.2022. wiki: <https://en.wikipedia.org/wiki/GPT4-Chan>.
- [2] Antonis Papasavva, Savvas Zannettou, Emiliano De Cristofaro¹, Gianluca Stringhini, and Jeremy Blackburn. Raiders of the Lost Kek: 3.5 Years of Augmented 4chan Posts from the Politically Incorrect Board. <https://arxiv.org/abs/2001.07487>, Release: 21.1.2020. Dataset available at <https://zenodo.org/record/3606810>.
- [3] Microsoft. Phi-2 Transformer. <https://huggingface.co/microsoft/phi-2>, Release: 13.12.2023. Accessed: 28.05.2025.
- [4] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, and Weizhu Chen Lu Wang. LoRA: Low-Rank Adaptation of Large Language Models. <https://arxiv.org/abs/2106.09685>, Release: 13.6.2021.
- [5] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-Resolution Image Synthesis With Latent Diffusion Models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 10684–10695, June 2022.
- [6] Christoph Schuhmann, Romain Beaumont, Richard Vencu, Cade Gordon, Ross Wightman, Mehdi Cherti, Theo Coombes, Aarush Katta, Clayton

- Mullis, Mitchell Wortsman, Patrick Schramowski, Srivatsa Kundurthy, Katherine Crowson, Ludwig Schmidt, Robert Kaczmarczyk, and Jenia Jitsev. LAION-5B: An open large-scale dataset for training next generation image-text models. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, editors, *Advances in Neural Information Processing Systems*, volume 35, pages 25278–25294. Curran Associates, Inc., 2022.
- [7] Patrick von Platen, Suraj Patil, Anton Lozhkov, Pedro Cuenca, Nathan Lambert, Kashif Rasul, Mishig Davaadorj, Dhruv Nair, Sayak Paul, William Berman, Yiyi Xu, Steven Liu, and Thomas Wolf. Diffusers: State-of-the-art diffusion models. <https://github.com/huggingface/diffusers>, 2022.
- [8] Lukas Biewald. Experiment Tracking with Weights and Biases, 2020. Software available from wandb.com.
- [9] Sondre Losvar and Roberto Neglia. PePeAI. <https://github.com/RobertoNeglia/PePeAI>, Release: 04.04.2025.
- [10] Sylvain Gugger, Lysandre Debut, Thomas Wolf, Philipp Schmid, Zachary Mueller, Sourab Mangrulkar, Marc Sun, and Benjamin Bossan. Accelerate: Training and inference at scale made simple, efficient and adaptable. <https://github.com/huggingface/accelerate>, 2022.
- [11] Nataniel Ruiz, Yuanzhen Li, Varun Jampani, Yael Pritch, Michael Rubinstein, and Kfir Aberman. Dreambooth: Fine tuning text-to-image diffusion models for subject-driven generation. 2022.

A Appendix

Other examples of generated posts and images can be found in the following figures. The images are of varying quality, but they all reflect the text in some way. The text is also of varying quality, but they all seem to be coherent with the topic and style of posts that could be found on /pol/. Some posts are more edgy than others, but they all seem to be coherent with the topic and style of posts that could be found on /pol/.



Figure 10: Police Pepe



Figure 11: Climate Change Pepe

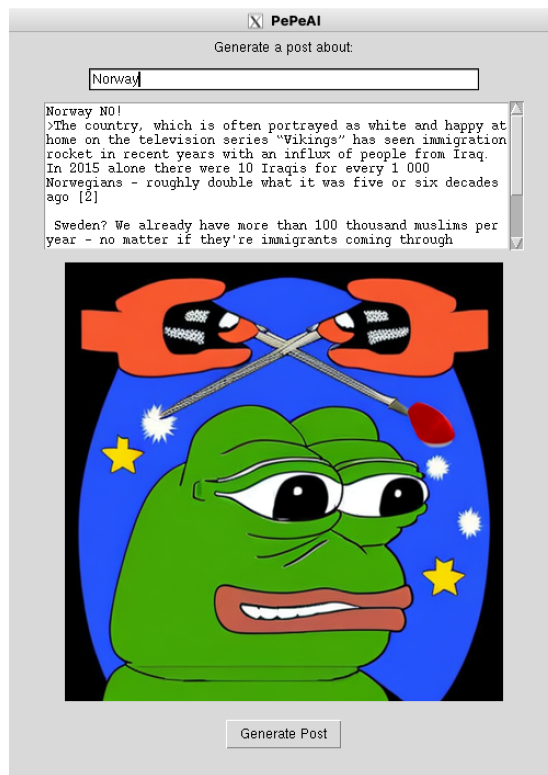


Figure 12: Norway Pepe